CoStream: Codec-Guided Resource-Efficient System for Video Streaming Analytics

Yulin Zou
Nanyang Technological University
Singapore
yulin001@e.ntu.edu.sg

Yan Chen
Beihang University
China
chenyan2022@buaa.edu.cn

Wenyan Chen
Nanyang Technological University
Singapore
wenyan.chen@ntu.edu.sg

JooYoung Park
Nanyang Technological University
Singapore
jooyoung001@e.ntu.edu.sg

Shivaraman Nitin
Institute of High Performance Computing
Singapore
shivaran@a-star.edu.sg

Luo Tao
Institute of High Performance Computing
Singapore
luotao@a-star.edu.sg

Francisco Romero
Georgia Institute of Technology
USA
faromero@gatech.edu

Dmitrii Ustiugov
Nanyang Technological University
Singapore
dmitrii.ustiugov@ntu.edu.sg

Abstract

Video streaming analytics is a crucial workload for vision-language model serving, but the high cost of multimodal inference limits scalability. Prior systems reduce inference cost by exploiting temporal and spatial redundancy in video streams, but they target either the vision transformer (ViT) or the LLM with a limited view, leaving end-to-end opportunities untapped. Moreover, existing methods incur significant overhead to identify redundancy, either through offline profiling and training or costly online computation, making them ill-suited for dynamic real-time streams.

We present CoStream, a codec-guided streaming video analytics system, built on a key observation that video codecs already extract the temporal and spatial structure of each stream as a byproduct of compression. CoStream treats this codec metadata as a low-cost runtime signal to unify optimization across video decoding, visual processing, and LLM prefilling, with transmission reduction as an inherent benefit of operating directly on compressed bitstreams. This drives codec-guided patch pruning before ViT encoding and selective key-value cache refresh during LLM prefilling, both of which are fully online and do not require offline training. Experiments show that CoStream achieves an improvement in throughput of up to 3×, and a reduction of up to 87% in GPU compute over state-of-the-art baselines, maintaining competitive accuracy with only 0∼8% F1 drop.

1 Introduction

Video streaming analytics [8, 16, 35, 42] has become increasingly indispensable across diverse domains, including surveillance [20, 86], traffic control [58, 94], retail operations [46, 80], and industrial automation [27, 53]. Driving this trend is the explosive growth of video sensing infrastructure: surveillance alone accounts for over 1.1 billion cameras deployed globally, with the installed base growing by more than 38% between 2020 and 2024 [39, 48]. The resulting volume of video data far exceeds what manual monitoring can handle, requiring models capable of reasoning across long, multimodal contexts. Traditional CNN-based approaches are inherently limited in capturing long-range temporal dependencies [3, 6, 65], while recent vision-language models (VLMs) [12, 30, 62, 70] have demonstrated strong potential for video understanding, making them a natural foundation for these workloads.

Deploying VLMs for continuous video analytics, however, is computationally demanding. A typical VLM-based pipeline consists of three stages: (1) Video streams are transmitted from edge cameras to a cloud server, where they are decoded into raw frames by video codecs. (2) A vision transformer (ViT) encoder partitions each frame into fixed-size spatial patches, encodes each patch into a visual token, and projects the resulting tokens into the LLM's embedding space. (3) The LLM processes these visual tokens together with a text query to produce the final inference output. To preserve temporal context in continuous streams, the pipeline operates over a sliding window of frames that advances by a small stride at each step [18, 31, 83, 86]. While this pipeline delivers powerful multimodal reasoning, it places substantial demands on GPU resources.

This high per-stream cost multiplies with the sheer number of deployed cameras. Across major cities, the number of CCTVs exceeds available GPUs by 8∼25× [14, 43, 44], creating a fundamental throughput bottleneck: the available GPU capacity is insufficient to sustain real-time processing across all concurrent streams. To pinpoint the performance bottlenecks in the VLM pipeline, we perform a

latency breakdown analysis of video analytics workloads in § 2.2, which reveals that transmission, visual processing, and LLM prefilling account for the majority of the end-to-end latency. A key underlying driver is the substantial spatiotemporal redundancy in video streams: consecutive frames share most of their content, and the overlapping sliding windows cause the pipeline to repeatedly compute largely identical visual context.

While recent systems have explored techniques to reduce redundant computation in the ViT encoder or the LLM decoder [22, 51, 59, 66], they suffer from two limitations. First, they optimize individual pipeline components in isolation, foregoing the holistic end-to-end gains that a unified approach could achieve. Second, several rely on expensive offline profiling [22, 51], producing optimization policies that cannot adapt to the continuously varying content and motion patterns of live streams. Both limitations leave end-to-end efficiency gains largely unrealized.

Video codecs, designed for efficient video storage and transmission [54, 67], capture this redundancy precisely as a byproduct of compression. Through inter-frame prediction, they store only motion vectors and residuals rather than full frames, as consecutive frames typically share over 95% of their pixel content. Prior video analytics systems have exploited codec signals only for preprocessing or indexing [1, 23, 81], yet VLM serving systems universally assume decoded frame input, leaving codec metadata entirely unexploited. These same signals offer an opportunity to guide both visual processing and LLM prefilling stages online and without offline training, but doing so is fundamentally challenging. First, codec primitives are defined in the compressed domain (e.g., macroblocks, motion vectors, and residual blocks), while VLMs operate on patches, tokens, and semantic representations; bridging this gap requires converting compressed-domain signals into optimization decisions without compromising accuracy. Second, Key-Value Cache (KVC) retention in LLM prefilling is inherently semantics-sensitive: as the sliding window advances, overlapping content may remain visually unchanged, yet its contextual role can shift, making naive state reuse semantically invalid. A practical system must therefore determine when state reuse preserves semantic fidelity and when refresh and correction are necessary, while keeping decision overhead low enough to preserve the latency gains.

We present CoStream, a Codec-guided Streaming video analytics system built on a key insight: codec metadata is not merely a byproduct of compression, but a low-cost runtime signal that can be exploited to co-design video decoding, visual processing, and LLM prefilling around a single unified metadata extraction pass. First, it integrates hardware-accelerated codec processing with single-pass decoding to eliminate redundant decoding across overlapping windows while extracting compressed-domain metadata at runtime. Second, it adopts a codec-guided token-pruning policy to identify motion-dynamic regions and prune redundant patches before ViT encoding. Third, CoStream leverages the same metadata to drive selective KVC refresh in the LLM stage, refreshing only drift-sensitive states while retaining the remaining cache entries with position correction. Together, these techniques systematically reduce redundant computation across visual processing and LLM prefilling while preserving temporal semantic consistency. We implement CoStream on top of vLLM [28] and evaluate it on two representative VLMs, showing up to 3× latency reduction (equivalently, 3× throughput improvement) and up to 87% GPU compute reduction over state-of-the-art baselines such as Déjà Vu [22] and VLCache [51], with only 0~8% F1 drop across the two evaluated VLMs. Our main contributions are as follows:

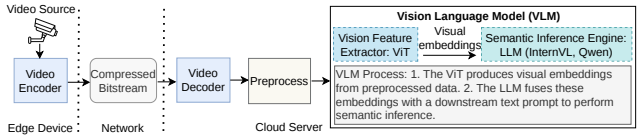


Figure 1. End-to-end serving pipeline for video streaming analytics: Video compression, bitstream transmission, decompression, preprocessing, and VLM inference (vision feature extractor (ViT) and semantic inference engine (LLM)).

- We characterize the bottlenecks of streaming VLM serving and identify the opportunities and challenges of exploiting codec metadata holistically across transmission, visual processing, and LLM prefilling.
- We present CoStream, a codec-guided streaming video analytics system that jointly optimizes all three stages via two inference optimizations: patch pruning before ViT encoding and selective KVC refresh, both driven by codec signals extracted once at video decode time, with transmission reduction as an inherent benefit.
- We implement CoStream on top of commodity video decoding hardware and the production vLLM serving framework, showing that CoStream reduces the end-to-end latency by up to 3× across multiple VLMs while preserving F1 within 0~8% drop of the baselines.

2 Background and Motivation

We begin with an overview of modern video streaming applications and their supporting systems. We then identify the challenges caused by growing streaming demands under limited compute resources. Finally, we highlight the opportunity to improve scalability and reduce latency by integrating codec metadata into the inference serving pipeline.

2.1 Video Streaming Analytics Systems

A representative video streaming serving pipeline is shown in Fig. 1. Surveillance cameras at the edge compress and transmit video streams over the network to a cloud server, which decodes the bitstream into raw frames. Each frame

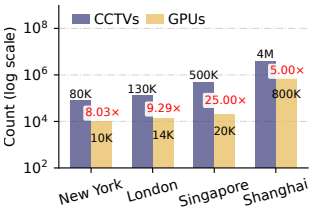


Figure 2. Statistics [14, 43, 44] of the imbalance between CCTVs and GPUs in different regions.

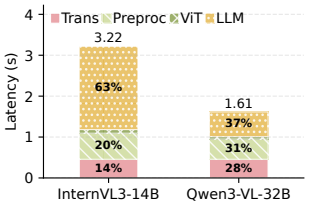


Figure 3. Latency breakdown. InternVL3/Qwen3-VL evaluated on 2/4 A100 (40GB) GPUs.

is preprocessed and partitioned into visual patches, which are fed into a ViT. The resulting patch embeddings are then passed through a projection module to produce a compact sequence of visual tokens. The LLM then fuses these tokens with a text prompt to perform semantic inference and generate a response.

Anomaly detection over live streams is a typical video analytics workload [33, 69, 74]. In a standard deployment, the system continuously analyzes incoming video streams by partitioning them into temporal windows and applying semantic queries to each window. As illustrated in Fig. 1, each segment is processed by a VLM-based pipeline, where the model evaluates a query such as: *"Describe the frames and determine if they show any abuse. Start your response with 'Yes' or 'No'."* Whenever a window is classified as "Yes", the system raises an alert for downstream action, thereby reducing the cognitive burden on human operators.

2.2 Can Today's Systems Keep up with the Load?

Current VLM serving systems are usually executed on high-performance GPU clusters in the cloud, which are shared across many video streams. However, in an urban environment, the number of CCTVs far exceeds the number of available GPUs [14, 43, 44], as shown in Fig. 2. For example, even if the city of London used all available GPUs in the area only for surveillance footage analysis, the mismatch would be dramatic: there are around 130k cameras, but only 14k GPUs [14, 43, 44]. This disparity reflects that for city-wide analytics to be viable, each GPU must handle an immense volume of data that far exceeds current hardware capability.

Consider a streaming video analytics system that uses a sliding window to preserve temporal context [18, 31, 83, 86]. The system partitions each stream into windows of size w and advances the window by a stride $s < w$ at each step. The stride is the temporal offset between consecutive windows; adjacent windows overlap by $w-s$. Based on the observation that 90% of urban crime events conclude within 40 s [85], we use a 40 s window in our analysis. We set the stride to 8 s (20% of the window), which provides the best latency-accuracy tradeoff in our sensitivity study (§ 6.3), and adopt a sampling rate of 2 FPS [22]. Under this configuration, each new 8 s of video triggers a re-process of the previous window's last 32 s. Consequently, a naive sliding-window design incurs up to 5× the computation of a reuse-aware design that processes only newly arrived content while preserving temporal context.

Table 1. Comparison with existing VLM optimized systems.

Method	Optimization Scope		Deployment Efficiency	
	ViT	LLM	No Train	Online
Default VLM	×	×	✓	×
Déjà Vu [22]	✓	×	×	×
CMC [59]	✓	×	✓	×
VLCache [51]	×	✓	×	×
CoStream (Ours)	✓	✓	✓	✓

To ground this demand in real-world performance, our analysis in Fig. 3 shows that serving a single video stream with such a window on two A100 (40GB) GPUs using InternVL3-14B [12] incurs a total latency of up to 3.2s. Under these conditions, a single A100 GPU can barely sustain 0.6 streams in real-time (2 GPUs / 3.2s ≈ 0.6 streams / GPU). Scaling this to Singapore's 500,000 CCTVs would theoretically require ~800,000 A100 GPUs, which is a staggering 40× the size of the city's current 20,000-unit GPU pool. This massive hardware requirement highlights a critical VLM serving efficiency opportunity: Naive serving strategies redundantly recompute overlapping temporal context across continuous streams. To make large-scale video analytics viable without prohibitive hardware costs, it is imperative to design serving systems that efficiently reuse temporal context and eliminate redundant computation.

Latency breakdown. To identify the primary bottlenecks, we profile a representative baseline pipeline in which the client transmits sampled JPEG frames to a vLLM server for VLM inference, at a representative edge uplink rate of 5Mbps [68] using two VLMs: InternVL3-14B [12] and Qwen3-VL-32B-Instruct [62]. As shown in Fig. 3, end-to-end latency is primarily driven by transmission (up to 28%), visual processing (including preprocessing and ViT encoding, up to 31%), and LLM prefilling (up to 63%). With frames transmitted individually as JPEGs, transmission overhead is significant under limited edge bandwidth. Within visual processing, CPU-bound preprocessing dominates, lacking the parallelism needed to handle continuous frame throughput. The prefill stage is the most expensive: even with InternVL3's internal 4× spatial compression, each 448 × 448 frame requires 256 tokens, injecting 20,480 visual tokens per 40 s window. Because consecutive frames share substantial content and sliding windows overlap heavily, the VLM repeatedly processes largely identical tokens [19]. These observations motivate CoStream, which targets all three bottlenecks through a unified approach that reduces redundancy across transmission, visual processing, and LLM prefilling simultaneously.

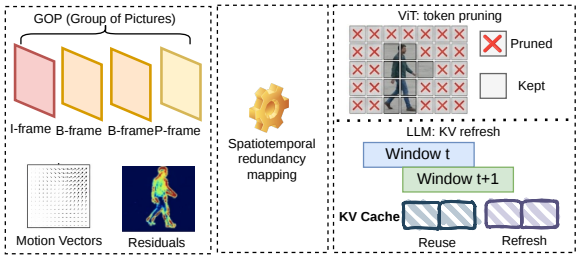


Figure 4. Overview of the codec-guided opportunities.

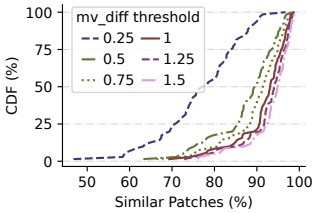


Figure 5. CDF of the similar patch ratio per frame across UCF-Crime at different MV thresholds (`mv_diff`).

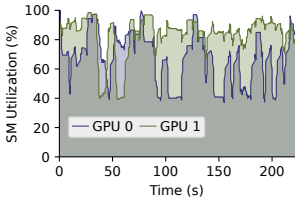


Figure 6. SM utilization trend of video stream inference with InternVL3-14B on 2 A100 GPUs of TP=2.

2.3 Limitations of the Existing Systems

To address the above inefficiencies, recent systems have proposed techniques to reduce redundant computation in both the ViT encoder and LLM. However, these approaches exhibit several limitations when applied to streaming video analytics workloads, as summarized in Table 1.

Limitation#1: ViT-centric optimizations ignore the dominant language decoder/prefill bottleneck. Most existing systems primarily target the ViT encoder—e.g., by pruning or reusing patches to reduce ViT computation [22], or by leveraging hardware acceleration for faster ViT execution [59]. CMC [59] accelerates ViT inference via a custom hardware–software co-design that shifts expensive spatiotemporal redundancy detection from the network to the codec, yet it does not address the overlapping redundancy induced by sliding-window inference in the *LLM prefill* stage. Déjà Vu [22] similarly avoids redundant ViT computation by identifying and reusing similar patches across frames, but leaves LLM prefill and decoding unchanged. Although these techniques can substantially reduce visual-encoder latency, they overlook the language decoder, which our measurements indicate is the dominant end-to-end bottleneck for many VLMs. Consequently, overall latency remains high because LLM prefill costs are not optimized.

Limitation#2: High operational overhead due to training or offline profiling requirements. Other researchers rely on heavy offline profiling to define policies that dictate which tokens are reused or pruned. This inherently increases deployment and maintenance complexity [22, 51, 75]. For example, Déjà Vu [22] requires additional training to learn patch-reuse policies, incurring extra data and compute costs while potentially degrading robustness under domain shift. Likewise, VLCache [51] relies on offline profiling to determine layer-wise recomputation ratios, and this profiling must be repeated across models, resolutions, and window configurations. Critically, streaming video analytics is inherently non-stationary because motion patterns, scene dynamics, lighting conditions, and event characteristics can change over time, rendering static, offline-derived policies brittle and often necessitating frequent re-training or re-profiling. Such operational overheads hinder practical adoption in real-world deployments that require online adaptability with minimal tuning effort.

2.4 Opportunities and Challenges

To overcome the above limitations, we seek opportunities for holistic optimization across visual encoding and LLM prefilling, while introducing minimal operational overhead. The key insight is that video streams exhibit substantial temporal redundancy: consecutive frames share much of their content, e.g., static background and predictable camera motion, and overlapping sliding windows cause the pipeline to repeatedly recompute largely identical visual context. Yet existing VLM serving systems operate entirely on decoded frames, overlooking the signals that the codec already derives to precisely capture this redundancy and could guide optimization across all stages.

2.4.1 Unlocking Optimization Opportunities with Video Codecs. Video codecs [54] offer a natural mechanism for addressing the bottlenecks above. As shown in Fig. 4, codecs organize raw video into Groups of Pictures (GOPs) with hierarchical I, P, and B frames. By exploiting spatial and temporal redundancy, this structure sharply reduces data volume and compresses raw video bitrate by orders of magnitude. Prior studies report compression ratios of 100:1∼270:1 in medical video [7] and up to roughly 1600:1 in surveillance-oriented face recognition workloads [26].

Beyond compression, predictive coding also exposes lightweight metadata, notably motion vectors (MVs) and residuals, that can serve as useful proxies for content changes. Prior work has shown that such signals can help eliminate redundant computation in visual pipelines [22]. MVs capture block-level displacement, while residuals quantify the remaining prediction error after motion compensation. By parsing these pre-existing signals at runtime, the system obtains low-overhead guidance for identifying regions that are likely reusable versus those that require recomputation. This creates two key opportunities: reducing redundant visual token computation and enabling downstream reuse across overlapping windows.

Opportunity#1: Codec-guided patch pruning in the ViT encoder. Building on the extracted signals discussed previously, we can aggregate both MVs and residuals into a unified spatial mask. By mapping this mask onto the ViT patch grid, the system can identify patches that are highly likely redundant. Specifically, regions with near-zero MVs and low residuals indicate static or predictable content and

can be safely pruned, while regions with large MVs or high residuals signal meaningful updates that require full computation. Since these signals are already available at runtime, the system can estimate temporal redundancy before visual encoding begins with negligible overhead.

To explore this opportunity, we analyze the distribution of MVs across the UCF-Crime dataset [15], as illustrated in Fig. 5. Our analysis reveals that a significant majority of patches exhibit minimal motion. Specifically, across all tested videos, 50% of frames contain patches that are 77%∼94% similar when evaluated under motion and residual thresholds. The high redundancy in streaming video imposes substantial GPU overhead on the ViT encoder. As shown in Fig. 6, even for a single video stream input with a 40-second window at 2 FPS on InternVL3-14B, the average SM utilization across two A100 GPUs reaches 52% and 67%, respectively. This indicates that a large fraction of GPU resources is occupied by redundant computation. Together, these observations show that a large fraction of GPU cycles is spent recomputing visual content that has barely changed between adjacent frames. This directly motivates our codec-guided patch pruning strategy: by using codec metadata to identify and skip redundant patches, the system reclaims wasted compute for regions with meaningful updates.

Opportunity#2: Codec-informed context-aware KVC refresh in the LLM decoder Sliding-window inference introduces substantial overlap across consecutive video segments, as shown in Fig. 7. While this overlap creates a structured KVC refresh opportunity, most existing KVC management methods are designed for generic memory efficiency, such as dynamic allocation, offloading, and cache reuse, rather than overlap-aware reuse under a shifting video context [28, 49, 78]. Although consecutive windows share overlapping visual content, the corresponding KV states in the LLM are not directly reusable. As the window advances, the overlapping tokens are placed under a different context, and their positions in the sequence may also shift. As a result, the hidden states of the same visual content can drift across windows in deep Transformer layers. Consequently, naively reusing cached KV states from the previous window can introduce semantic drift and degrade reasoning accuracy, while full recomputation remains expensive. We further observe that not all token drifts are equally important: some regions remain stable and insensitive to context changes, whereas others, such as motion-intensive regions or newly emerging events, require refreshed features. Codec metadata provides a lightweight signal to identify these drift-sensitive tokens, enabling a selective refresh strategy that updates only critical KV states while reusing the rest.

2.4.2 Challenges. While codec-guided optimizations present promising opportunities, realizing them in practice entails several challenges.

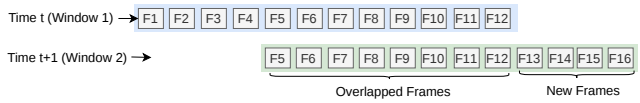


Figure 7. Illustration of overlapping redundancy in sliding-window VLM inference. When the window slides over the video stream (from Time t to $t+1$), a significant amount of frames (F5∼F12) are overlapped between adjacent windows.

C_1**: From codec primitives to model-compatible pruning decisions.** Although the aforementioned codec metadata in § 2.4.1 provides valuable optimization opportunities, it is inherently expressed in the units designed for video compression rather than model inference, such as I/P/B frames, motion vectors, and residual changes. In contrast, the VLM operates on ViT patches to produce semantic tokens. Bridging this representation gap presents a non-trivial system challenge. First, the system must accurately map block-level change signals to patch-level decisions under dynamic rescaling, cropping, and varying resolutions. Second, it must determine the optimal pruning aggressiveness without degrading downstream semantics. For example, regions with minimal motion and small residuals might still contain semantically critical cues such as subtle human gestures, slow-moving distant targets, or persistent background objects that are vital for long-term reasoning. Consequently, a robust system design must efficiently convert these noisy, low-level codec signals into model-compatible pruning policies. These policies must generalize across diverse video contents, GOP structures, and motion patterns while strictly maintaining a negligible decision overhead.

C_2**: Semantics-preserving partial refresh with position-sensitive decoding.** Selective KVC refresh is not a purely syntactic optimization: in sliding-window video analytics, advancing the window can change the decision semantics (e.g., event boundary interpretation and which evidence the model should attend to). Thus, previously computed intermediate states may no longer be valid for the current query context. The core difficulty is to design a reuse mechanism that is simultaneously (i) semantic-valid, (ii) position-consistent, and (iii) low-overhead. In particular, reused KV states are entangled with token positions and cross-token interactions inside the decoder; partial refresh can introduce subtle inconsistencies (e.g., mixing recomputed and reused states across layers/positions) that are hard to detect but can significantly affect outputs. Therefore, the system must determine both *where*, i.e., which spatial token regions, and *when*, i.e., at which window positions, the reused state remains valid, or a refresh is necessary, while ensuring that the decision overhead does not erase the latency gains.

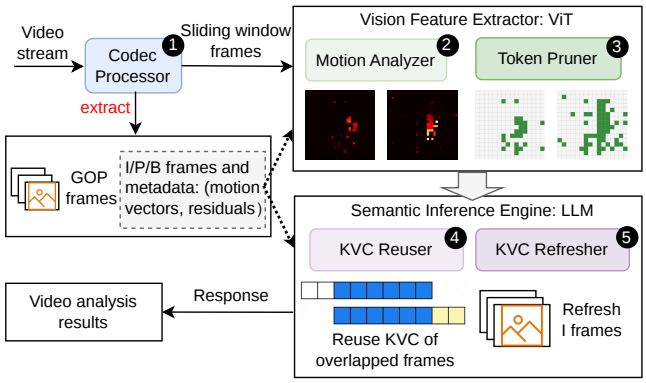


Figure 8. System architecture overview of CoStream.

3 System Design

We present CoStream: a codec-guided system for efficient streaming video analytics. CoStream addresses the challenges outlined in § 2.4.2 by optimizing across the whole analytics pipeline, specifically transmission, ViT encoder, and LLM decoder.

3.1 System Overview

Fig. 8 shows CoStream's architecture overview. The ***Codec Processor*** (❶ in Fig. 8) ingests encoded video streams from edge CCTV cameras, decoding frames while simultaneously extracting compressed-domain metadata such as motion vectors and residuals. This metadata provides essential cues for subsequent visual feature extraction. Specifically, the ***Motion Analyzer*** (❷ in Fig. 8) leverages these motion vectors to differentiate between stable and dynamic regions, enabling the ***Token Pruner*** (❸ in Fig. 8) to systematically prune redundant visual tokens.

The resulting visual tokens are then fused with textual tokens and passed to the *semantic inference engine*, which is built on an LLM. During the inference phase, the ***KVC Reuser*** (❹ in Fig. 8) manages Key-Value Caches (KVCs) across sliding windows, while the ***KVC Refresher*** (❺ in Fig. 8) selectively refreshes critical tokens for key frames to maintain temporal semantic consistency. Finally, the LLM generates a response by synthesizing the fused multimodal embeddings. CoStream's design emphasizes the tight integration between codec metadata and model computation, enabling effective elimination of redundant processing while minimizing accuracy loss in video understanding.

3.2 Hardware-Accelerated Codec Processing

Video streams typically arrive as compressed bitstreams, whose lower bitrate naturally reduces transmission (§ 2.4.1) and ingestion overhead before downstream analytics begin. While this compression benefit is provided by the codec itself, CoStream is explicitly designed to preserve and exploit the compressed representation through the front-end pipeline. The ***Codec Processor*** operates directly on the compressed stream using commodity GPU video engines [2, 45, 81], e.g., NVIDIA NVDEC, which are widely available across datacenter, consumer, and edge platforms. In a naive sliding-window design, overlapping windows cause the same frames to be decoded multiple times, once for each window they appear in. CoStream eliminates this redundancy by decoding the bitstream sequentially in a single pass and buffering the results, so that all overlapping windows share the same decoded frames without repeated decoding. During decoding, the ***Codec Processor*** also extracts codec metadata for downstream token-pruning decisions. Rather than relying on expensive pixel- or token-level analysis over decoded frames, CoStream uses compressed-domain metadata as a lightweight pruning signal. Because these signals are already embedded in the encoded stream, they provide a low-overhead representation of temporal dynamics and allow CoStream to avoid the cost of explicitly computing optical flow or other motion cues.

The decoded frames are preprocessed on the GPU without being transferred back to the CPU. Resizing, color-space conversion, and normalization are fused into a single batched operation over all frames. This design eliminates unnecessary CPU-GPU data movement, which would otherwise introduce substantial preprocessing overhead, directly addressing the bottleneck identified in § 2.2. The preprocessed frames are then streamed into a temporal buffer and organized into sliding windows for downstream ViT encoding. The previously-extracted codec metadata is also streamed for guiding later pruning decisions. Given a window size w and a stride s as defined in § 2.2, the k-th video window covers the interval $[ks, ks + w)$. By decoupling sequential codec processing from logical window formation, CoStream ensures that each frame is decoded only once, even if it appears in multiple overlapping windows. This design eliminates redundant codec work and improves the efficiency of online video analytics.

3.3 Motion Vector-Guided Token Pruning

Unlike prior systems [4, 29, 47] that compute cosine similarity or token-importance scores online, CoStream directly leverages motion vectors and residuals already available in P-frames, the dominant inter-coded frames in streaming pipelines (§ 2.4.1), as lightweight signals of region-level changes relative to a reference frame (I frame). Standard ViT encoders overlook this "hint" and redundantly compute features for static or predictable patches. To address this inefficiency, CoStream implements a motion vector-guided token pruning strategy that identifies and discards these redundant patches before they enter the ViT layers, bypassing unnecessary computation without the need for additional scoring operations. This addresses challenge C_1 in § 2.4.2.

3.3.1 Motion Vector Analysis. Motion vectors provide a compressed-domain signal of temporal variation by indicating how each coded block is predicted from a reference frame. For each block m in a P-frame at time t, the codec provides a motion vector $\mathbf{v}_t^m = (\Delta x, \Delta y)$ that specifies the offset

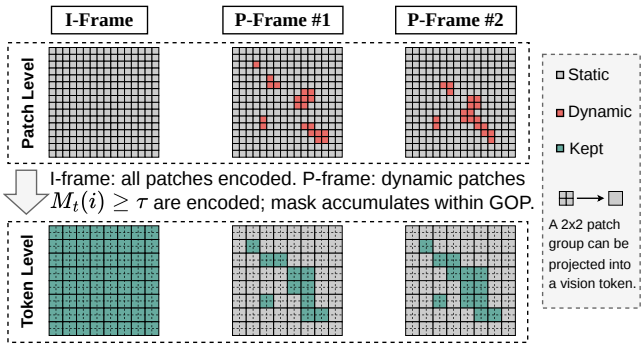


Figure 9. Illustration of the motion-guided token pruning policy. Each (2×2) group of neighboring patches is projected into a vision token. A token is retained if any of its constituent patches are marked dynamic, otherwise it will be discarded.

from the current block to its prediction region in the reference frame. We use its magnitude to quantify the degree of motion:

$$V_t^m = \|\mathbf{v}_t^m\|. \tag{1}$$

In addition to motion vectors, the codec also produces a residual signal that captures pixel-level differences after motion compensation. We quantify this signal for block m using the sum of absolute differences, a standard codec-internal distortion metric [59], computed between the pixel values of the current block B_t^m and its motion-compensated prediction $\hat{B}_t^m$:

$$R_t^m = \sum_{p \in B_t^m} \left| B_t^m(p) - \hat{B}_t^m(p) \right|. \tag{2}$$

To align codec signals with the ViT input layout, we resample the block-level motion and residual maps onto the patch grid, yielding $V_t^m = V_t(i)$ and $R_t^m = R_t(i)$ from block position m to patch position i. The ***Motion Analyzer*** then constructs a patch-level motion mask:

$$M_t(i) = V_t(i) + \alpha R_t(i), \tag{3}$$

where α controls the relative contributions of motion displacement and residual error.

In principle, both terms can improve the fidelity of motion estimation. In practice, however, motion vectors capture the primary variations in our target surveillance workloads, which are dominated by static backgrounds and relatively predictable motion. Moreover, hardware video decoders such as NVIDIA NVDEC expose only reconstructed frames and motion vectors, without providing residuals as accessible runtime outputs. Accordingly, our default hardware-decoded implementation sets $\alpha = 0$ and uses motion vectors alone as the pruning signal. We evaluate this design choice in § 6.3 and describe the details in § 4.

3.3.2 Token Pruning. Existing pruning methods [4, 52] determine token importance by analyzing feature maps or attention weights during inference, introducing computational overhead that scales with the number of input tokens and often offsets the speedup gained from pruning. CoStream eliminates this overhead by shifting the pruning decision entirely to the compressed domain before any ViT computation is performed, thus addressing challenge C_1 in § 2.4.2. Concretely, the ***Token Pruner*** classifies each patch as *dynamic* or *static* based on whether its motion magnitude M_t exceeds a threshold τ:

$$\text{dynamic}(i) = M_t(i) \geq \tau. \tag{4}$$

To preserve temporal consistency, the dynamic mask is accumulated within each GOP: the active set of a P-frame is defined as the union of its own detections and those of all preceding P-frames since the last I-frame. Thus, once a patch is marked dynamic, it remains active until the next I-frame resets the mask. I-frames are always fully encoded and provide the reference visual context for subsequent P-frames. This policy is illustrated in Fig. 9.

After ViT encoding, VLMs typically apply a spatial downsampling projection that groups neighboring patches into fewer visual tokens before passing them to the LLM. To preserve compatibility with this operator, CoStream expands the patch-level dynamic mask to a group-complete mask: if any patch within a spatial group is dynamic, all patches in that group are retained for encoding. CoStream then executes the ViT only on the selected patches, restores their outputs to the original spatial layout, and applies the native downsampling projection. Finally, only projected tokens corresponding to dynamic spatial groups are forwarded to the LLM. This design reduces both ViT computation and the LLM prefill sequence length while preserving the spatial grouping required by the downstream projector.

3.4 Selective KVC Refresh

Following token pruning, the VLM enters the LLM prefilling phase to construct the KVC for the current video window. As defined in § 2.2, a small stride relative to the window size achieves the best accuracy-latency tradeoff, meaning a large fraction of visual tokens overlap between consecutive windows. While full reuse of previous states can eliminate this overhead, it suffers from severe contextual drift, where stale KV states fail to align with evolving video content. This can lead to significant accuracy degradation. While full recomputation avoids this accuracy degradation, it wastes computation proportional to the overlap. To address challenge C_2 in § 2.4.2, CoStream implements a selective KVC refresh strategy that identifies and refreshes a small set of tokens more likely to require recomputation, while reusing and correcting the remaining cache entries. Rather than relying on attention-score divergence or offline-profiled recomputation ratios [51, 78], CoStream uses codec-derived frame-type information extracted by codec as a lightweight runtime signal for overlap-aware KV reuse.

3.4.1 Critical-Token KVC Refresh. Fig. 10 shows an example with GOP=4, each including 1 I-frame and 3 P-frames, with a window size of $w = 12$ frames and a stride

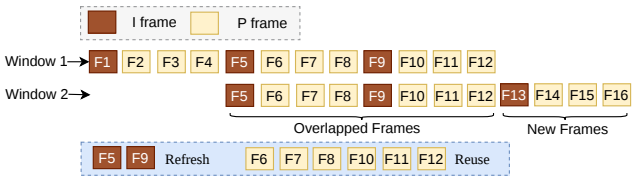


Figure 10. Example of selective KVC refresh with codec metadata.

of $s = 4$ frames (33%), operating at 1 FPS. When the window slides from the initial position ($F1 \sim F12$) to the next ($F5 \sim F16$), the raw pixels in the overlapping region ($F5 \sim F12$) remain identical. However, their KV states are not directly reusable because the VLM computes each token representation under the full multimodal context rather than in isolation. Specifically, the newly arrived content in the incoming frames ($F13 \sim F16$) reshapes the attention dependencies for the overlapping tokens, even though their visual appearance is unchanged. Consequently, naive KVC full reuse introduces significant approximation errors and degrades downstream reasoning quality. This creates a fundamental trade-off: while we must update KV states to maintain accuracy, recomputing the entire 12-frame window from scratch would waste 67% of the FLOPs on redundant visual data.

To balance efficiency and accuracy, CoStream refreshes only a small set of anchor tokens and reuses the rest. Tokens derived from I-frames serve as anchors as they provide stable reference content within each GOP and are most sensitive to context shifts as the window advances. We also ensure that the I-frame is the first frame in the overlapped region, anchoring the reused context at a stable boundary and reducing its susceptibility to attention sink [75]. The ***KVC Refresher*** recomputes their KV states under the new window context by feeding cached visual embeddings back into the LLM prefill path, without re-executing the ViT encoder. Non-anchor tokens from overlapping P-frames, which mainly capture local changes relative to nearby references, are reused after position correction, trading minor approximation error for significantly reduced prefilling cost.

3.4.2 Position-Consistent KVC Reuse. An additional challenge arises from the position-sensitive nature of KV states. As the window advances, reused tokens appear at different absolute positions from those in the previous window, which invalidates direct KV reuse. To restore positional consistency, CoStream applies RoPE-based position correction to the reused keys [78]. For a reused token j, let $p_{\text{old}}(j)$ and $p_{\text{new}}(j)$ denote its position embeddings in the previous window $t-1$ and current window t, respectively. CoStream updates the cached key K_t as:

$$\hat{K}_t(j) = R(p_{\text{new}}(j) - p_{\text{old}}(j))\ K_{t-1}(j), \tag{5}$$

where $\mathbf{R}(\cdot)$ denotes the rotary transformation. Intuitively, this operation "rotates" the existing key embedding to account for its new relative distance from other tokens in the sequence. This operation adjusts the reused key to remain consistent with the token's updated relative position in the current window. The value state is reused directly: $\hat{V}_t(j) = V_{t-1}(j)$, as value states do not explicitly encode positional information in RoPE-based attention and can therefore be reused directly in our design.

To further minimize latency, CoStream avoids the overhead of CPU-GPU memory transfers during cache updates. The ***KVC Reuser*** maintains the previous window's KV cache resident in GPU memory and performs these updates in-place. Once the overlapping tokens' KV states ($F5 \sim F12$ in Fig. 10) are thus corrected and refreshed, they are concatenated with the subsequently computed KV states of the incoming frames ($F13 \sim F16$) to construct the complete input for the current window.

At this point, the prefilling phase concludes, and the first token generation is triggered. By selectively refreshing anchor tokens while reusing and repositioning the rest, CoStream reduces redundant computation in the LLM prefill stage while maintaining useful contextual alignment in practice, as we evaluate in § 6.

4 Implementation Details

We implement CoStream on top of vLLM v0.11.0 [64], with model-specific extensions to support token pruning and sliding-window KVC management for VLM serving.

The ***Codec Processor*** is implemented as a front-end module in our VLM serving pipeline, comprising approximately 600 lines of Python code in our prototype. It ingests incoming compressed H.264 [67] video streams and performs codec-aware preprocessing before dispatching inputs to the VLM. Specifically, it partitions each stream into windowed clips and uses NVIDIA NVDEC for hardware-accelerated decoding. The ***Motion Analyzer*** and ***Token Pruner*** are integrated into the ViT encoder in vLLM, with approximately 3,000 lines of additional Python code in total. Since different VLMs adopt different ViT architectures and tokenization pipelines, we implement model-specific adaptations for each supported model family. These components use codec-derived motion information to identify redundant visual tokens and prune them before feature extraction.

We implement the ***KVC Reuser*** and ***KVC Refresher*** by extending LMCache v0.3.9 [13, 61] with 2,500 lines of Python code. Built on LMCache's cache-management primitives and chunk-based indexing, our implementation supports selective KVC refresh for sliding-window video inference. We extend its indexing and cache management logic to handle overlapping clips and GOP-aligned anchor selection for KVC refresh. We further add model-specific integration so that reused KV states match each model's transformer architecture and RoPE scheme.

5 Methodology

Testbed. All experiments are conducted on a high-performance server node equipped with four NVIDIA A100 (40GB, SXM4) GPUs running Linux 6.8.0-57-generic with

Table 2. Models and configurations used in evaluations.

Model	ViT Encoder	LLM Backbone	GPUs
InternVL3 [95]	InternViT (300M)	Qwen2.5-14B	2×A100
Qwen3-VL [63]	Qwen-ViT (600M)	Qwen3-32B	4×A100

Note: Values in parentheses (e.g., 300M and 600M) denote the number of parameters for each ViT encoder. Both InternVL3 and Qwen3-VL are served with tensor parallelism (TP=2 and TP=4, respectively)

CUDA 13.1. These GPUs are interconnected via third-generation NVLink. The system features an AMD EPYC 7713 64-Core CPU and 512GB of DDR4 system RAM, ensuring sufficient bandwidth for host-device data transfer.

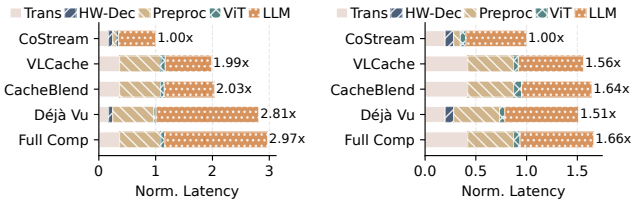
Models. We evaluate CoStream using two state-of-the-art VLMs, InternVL3 [95] and Qwen3-VL [63] to cover a diverse range of model architectures and scales (see Table 2).

Baselines. We compare CoStream against four baselines:

- **Full Comp**: An unoptimized VLM serving baseline implemented on top of vLLM [64], where every sampled frame is fully preprocessed and encoded by the ViT, and all resulting visual tokens are passed to the LLM without token pruning or KV-cache reuse.
- **Déjà Vu** [22]: A VLM query engine that specifically optimizes the ViT encoding. It reduces computation across consecutive frames through reusing similar patches and further translates FLOP savings into wall-clock speedups through joint memory-compute compaction.
- **CacheBlend** [78]: A KVC management scheme designed to accelerate the LLM prefill phase in RAG workloads. Unlike prefix caching, it supports reuse for non-prefix chunks by selectively recomputing a top-k subset of tokens to blend disparate KV caches and preserve accuracy.
- **VLCache** [51]: A multimodal cache reuse framework that accelerates the LLM prefill stage for recurring inputs. It avoids costly recomputation by caching both KV states and encoder features from prior multimodal inputs and by using a dynamic layer-aware strategy to balance efficiency and accuracy.

Dataset and Request Generation. Experiments are conducted on the UCF-Crime dataset [15], a collection of 1,900 untrimmed, real-world videos spanning a wide spectrum of scene dynamics, camera angles, and motion characteristics. Our evaluation uses a 40-second sliding window sampled at 2 FPS, requiring each video to be at least 60 s to ensure multiple overlapping windows; with an average duration of approximately four minutes, the vast majority of videos comfortably meet this threshold, yielding over 3,500 minutes of eligible footage. Each sliding-window segment is paired with a textual query asking whether it contains a target anomaly, and requests are replayed in a streaming fashion to emulate online video analytics serving

Metrics. We evaluate CoStream along three dimensions: accuracy, latency, and resource efficiency. For accuracy, we report Precision, Recall, and F1-score at the video level by aggregating predictions across the windows of each video against the ground truth labels. Specifically, an anomalous video is labeled as a True Positive if at least two consecutive windows produce a positive response, and as a False Negative otherwise; the inverse applies to normal videos. To quantify speedup, we measure stage-wise latency. Finally, we assess resource efficiency using the number of tokens after pruning and the corresponding FLOPs.



(a) InternVL3 **(b)** Qwen3-VL

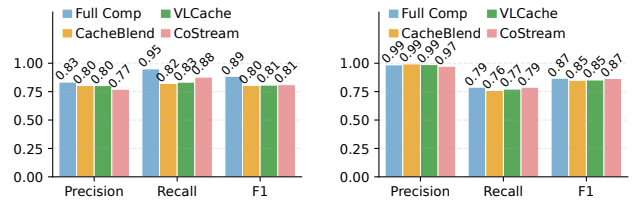
Figure 11. Latency speedup of CoStream. `HW-Dec` refers to the stage with hardware-accelerated codec decoding.

6 Evaluation

In this section, we evaluate CoStream along several key dimensions. We measure its latency and accuracy across different models, perform an ablation study to quantify the contribution of each component, conduct a sensitivity analysis for key parameters, and measure the runtime overhead of CoStream to ensure that these optimizations do not offset the overall gains. Unless otherwise specified, the end-to-end and component-level experiments use the parameter configuration selected from the sensitivity analysis in § 6.3: a stride of 20% of the window size, an MV threshold of 0.25 pixel, and a GOP size of 16 frames.

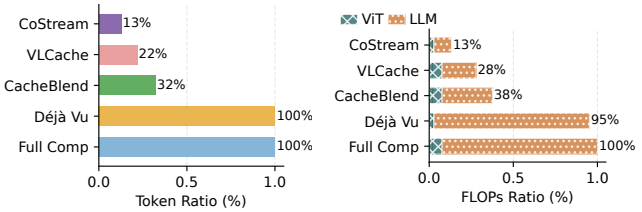
6.1 End-to-End Performance

6.1.1 Latency Speedup. To evaluate CoStream's end-to-end latency speedup across the video analytics pipeline, we break down the total latency into transmission, codec decoding, preprocessing, ViT execution, and LLM inference. Fig. 11 reports the results. For InternVL3, CoStream achieves up to 2.97× speedup over *Full-Comp*, while for Qwen3-VL, it achieves 1.66× speedup. These latency speedups translate into up to 3× higher effective processing throughput under the same hardware budget, assuming sequential processing of streams on a single GPU in § 2.2). Breaking the gains down by stage, CoStream reduces transmission latency by 2.12×, confirming the benefit of codec compression in lowering data transfer cost. For preprocessing and ViT execution, CoStream achieves 7.42× speedup for InternVL3 and 4.18× for Qwen3-VL relative to *Déjà Vu*, demonstrating the effectiveness of GPU-based acceleration and codec-guided token pruning. For LLM prefilling, CoStream further delivers up to 1.35× speedup over *CacheBlend* and 1.25× over *VLCache*, validating the effectiveness of efficient selective KVC refresh in reducing redundant computation.



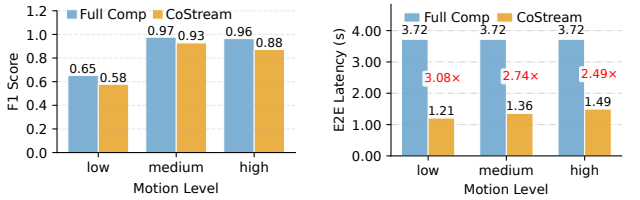
(a) InternVL3 **(b)** Qwen3-VL

Figure 12. Precision, Recall and F1 Score of CoStream.



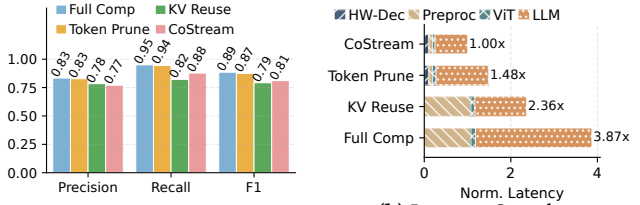
(a) Memory (tokens) required **(b)** Compute (FLOPs) required

Figure 13. Memory and compute resource savings of CoStream with InternVL3.



(a) F1 Score **(b)** Latency Speedup

Figure 14. Performance across video motion intensity levels with InternVL3.



(a) Accuracy **(b)** Latency Speedup

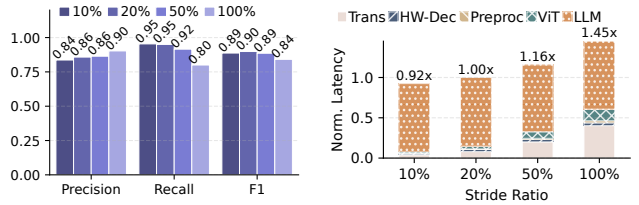
Figure 15. CoStream's component performance contributions with InternVL3.

Our end-to-end gains combine two sources: (1) front-end improvements from compressed-stream ingestion, hardware decoding, and GPU preprocessing, and (2) inference-stage savings from codec-guided token pruning and selective KVC refresh. We include both because CoStream is designed as an end-to-end streaming serving system rather than an isolated model-side optimization. Accordingly, comparisons to prior work should be interpreted as end-to-end system comparisons, while the stage-wise breakdown in Fig. 11 shows where the gains arise.

6.1.2 Accuracy. We also evaluate whether CoStream preserves semantic accuracy under codec-guided token pruning and selective KVC refresh. Fig. 12a and Fig. 12b report the average Precision, Recall, and F1 scores across all crime categories for InternVL3 and Qwen3-VL, respectively. CoStream maintains accuracy close to *Full-Comp* on both models, with only modest F1 degradation, from 0.89 to 0.81 for InternVL3 and even zero degradation for Qwen3-VL. CoStream also remains competitive with *VLCache* and *CacheBlend* on both models. Overall, these results show that CoStream preserves most of the semantic fidelity required for accurate inference while substantially reducing computation, validating codec-guided pruning and selective KVC refresh as effective optimizations with limited accuracy degradation.

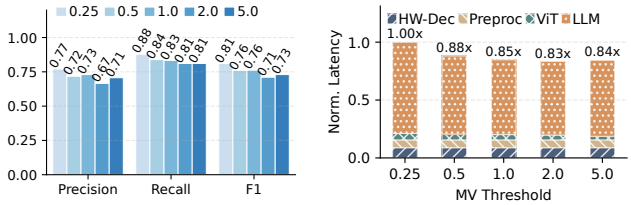
6.1.3 Resource Savings. By reducing the number of visual tokens processed by the ViT and eliminating redundant computation in the LLM prefilling stage, CoStream also delivers substantial resource savings. We quantify this benefit in terms of total processed tokens and consumed FLOPs. Fig. 13a shows that, across all tested video clips, CoStream achieves average token reductions of 85%, 60%, and 40% relative to *Full-Comp*, *CacheBlend*, and *VLCache*, respectively, across the two models. This reduction is also reflected in lower compute demand in both stages. As shown in Fig. 13b, CoStream reduces total FLOPs by an average of 87% for InternVL3. Overall, these results show that CoStream not only reduces latency but also significantly lowers computational demand, improving resource efficiency and enabling higher throughput or deployment on constrained GPU resources. Since these savings depend in part on how much codec-guided pruning is exposed by a video's motion characteristics, we next break the results down by motion level.

6.1.4 Performance Across Motion Levels. The aggregate results above demonstrate strong overall gains; we next analyze how these gains vary across motion levels, defined by partitioning the test videos into three equal-sized groups (low, medium, and high) based on average motion-vector magnitude. CoStream achieves 3.08×, 2.74×, and 2.49× speedup on low-, medium-, and high-motion videos, respectively. This trend broadly follows the pruning ratio: codec-guided token pruning removes 50%, 27%, and 13% of visual tokens in the three groups. Lower-motion videos expose more redundancy, while higher-motion videos expose less. At the same time, CoStream remains effective even in the high-motion group, delivering a 2.49× speedup with only 13% of tokens being pruned, indicating that selective KVC refresh provides a motion-independent source of savings by reusing and refreshing KV states regardless of how much token pruning achieves. For accuracy, CoStream shows limited and relatively stable F1 degradation across motion levels (0.08, 0.04, and 0.07 for high-, medium-, and low-motion videos), confirming that accuracy remains stable even as pruning grows more aggressive at lower motion levels. Overall, these results show that CoStream remains effective across substantially different motion regimes, maintaining substantial speedups even at high motion while keeping accuracy loss small and uniform.



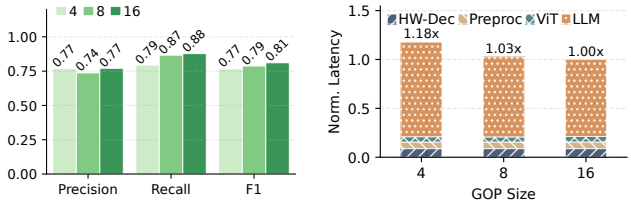
(a) Accuracy **(b)** Norm. Latency

Figure 16. Sensitivity analysis of stride ratio.



(a) Accuracy **(b)** Norm. Latency

Figure 17. Sensitivity analysis of MV threshold.



(a) Accuracy **(b)** Norm. Latency

Figure 18. Sensitivity analysis of GOP.

6.2 Per-Component Efficiency Analysis

To understand the contribution of each optimization component in CoStream, we perform an ablation study in which we selectively enable individual components on top of the vanilla baseline (*Full-Comp*) and measure the resulting accuracy and latency. Fig. 15 reports the results for InternVL3. Both components improve efficiency, but they contribute differently to the overall latency–accuracy tradeoff. Codec-guided token pruning alone achieves a 2.61× speedup with only a small F1 drop from 0.89 to 0.87, indicating that it captures much of the redundant visual computation while largely preserving accuracy. By contrast, selective KVC refresh alone provides a 1.64× speedup but reduces F1 to 0.79, showing that it contributes additional latency reduction at a larger quality cost. When combined, these optimizations achieve a 3.87× speedup with an F1 of 0.81, further amplified by GPU-based decoding and preprocessing. These results show that the two components are complementary: codec-guided token pruning targets spatial redundancy within individual frames before ViT encoding, while selective KVC refresh targets temporal redundancy across overlapping windows during LLM prefilling. The ablation shows that codec-guided token pruning provides most of the accuracy-preserving speedup, whereas selective KVC refresh contributes additional latency reduction, but also drives most of the quality tradeoff.

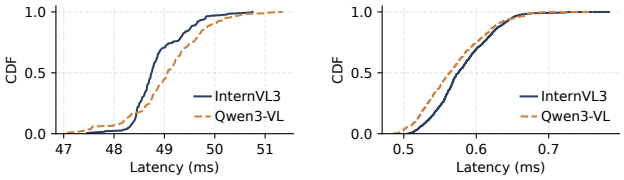
6.3 Sensitivity of Key Parameters

To identify optimal settings for these parameters, we perform a sensitivity analysis by varying each parameter while keeping the others fixed with InternVL3.

6.3.1 Stride Ratio. To identify an appropriate stride ratio for sliding-window inference in the crime detection task, we perform a sensitivity analysis by varying the stride from 10% to 100% of the window size and measuring the resulting accuracy and latency. Fig. 16 reports the results. Overall, smaller strides improve detection quality by updating the VLM input more frequently, which reduces the chance of missing temporally continuous events that span window boundaries. As a result, reducing the stride from 100% (no overlap) to 20% improves the F1 score from 0.84 to 0.89. Interestingly, the trend is not strictly monotonic: a 10% stride yields slightly lower accuracy than a 20% stride. We conjecture that overly small strides introduce excessive overlap across adjacent windows, which can amplify prediction noise and lead to diminishing, or even negative, returns in semantic accuracy. For latency, larger strides increase per-inference cost because less overlap between consecutive windows reduces KVC reuse, forcing more tokens to be recomputed during prefill. Relative to the chosen 20% stride, a 100% stride incurs 1.45× higher latency per inference, while a 10% stride reduces latency marginally to 0.92× at the cost of lower accuracy. Since 20% achieves the highest F1 of 0.90 with favorable per-inference latency, we adopt it as the default stride ratio in CoStream.

6.3.2 MV Threshold. To evaluate the effect of the MV threshold on the accuracy-latency trade-off, we vary it from 0.25 to 5.0 pixels. The MV threshold controls the aggressiveness of codec-guided token pruning by determining which regions are treated as static. As shown in Fig. 17, increasing the MV threshold makes pruning more aggressive, reducing normalized latency from 1.00× to 0.83× but also degrading the F1 from 0.81 to 0.73. Conversely, a smaller threshold preserves more tokens and thus maintains higher accuracy, at the cost of smaller efficiency gains. We therefore use MV=0.25 in the remaining experiments, as it provides the best accuracy-efficiency balance: it achieves the highest F1 (0.81) in this sensitivity study and, when used in the full pipeline, still yields up to 2.97× end-to-end latency reduction over *Full-Comp* (Fig. 11).

6.3.3 GOP Size. To evaluate the effect of GOP size on the accuracy-latency tradeoff, we vary it among 4, 8, and 16 frames. The GOP size determines I-frame frequency and thus affects both KV reuse opportunities and refresh overhead. As shown in Fig. 18, both accuracy and latency improve monotonically with larger GOP sizes. For latency, larger GOP sizes reduce I-frame recomputation frequency,



(a) Token Pruning **(b)** KVC Refresh

Figure 19. System overheads.

with GOP=4 incurring 1.33× the latency of GOP=16. For accuracy, smaller GOP sizes lead to more frequent KVC refreshes that disrupt temporal continuity across the sliding window, preventing the LLM from accumulating stable cross-frame context; F1 scores are 0.77, 0.79, and 0.81 for GOP sizes 4, 8, and 16, respectively. Since GOP=16 achieves both the lowest latency and the highest accuracy among the tested settings, we adopt it as the optimal configuration.

6.4 System Overhead

We quantify the runtime overhead of CoStream's token selection before ViT encoding and during KVC refresh. As shown in Fig. 19, both operations introduce only modest overhead. For InternVL3, token pruning and KVC refresh incur average/max overheads of 48.9/50.8 ms and 0.6/0.8 ms per request, respectively; for Qwen3-VL, the corresponding overheads are 49.1/51.4 ms and 0.6/0.8 ms. Notably, scaling to the much larger Qwen3-VL increases overhead only marginally. In both cases, the combined overhead of about 50 ms accounts for just 3.9% and 4.5% of CoStream's optimized end-to-end latency, respectively. Even with this overhead, CoStream remains up to 2.97× faster than *Full-Comp* across both models. These results show that CoStream's optimization logic is lightweight, and its efficiency gains are not offset by the added overhead.

6.5 Scope, Applicability, and Portability

CoStream is designed for continuous video analytics workloads where a VLM processes streams over sliding windows. Its optimizations exploit two properties inherent to such workloads: temporal redundancy between frames (addressed by codec-guided token pruning) and across overlapping windows (addressed by selective KVC refresh). Since neither mechanism depends on task-specific semantics, CoStream generalizes beyond surveillance footage to any sliding-window VLM setting with recurring visual context, whether sourced from dashcams, drones, broadcast streams, or in-store cameras.

We evaluate CoStream on two architecturally distinct VLMs, InternVL3 and Qwen3-VL, to demonstrate its model-agnostic design. Patch pruning operates upstream of the vision backbone and requires only knowledge of the input patch layout, while selective KVC refresh integrates at the LLM prefill stage through standard positional encoding schemes such as RoPE. Adapting CoStream to a new VLM primarily involves a one-time integration of the token layout and position correction logic, as detailed in § 4.

Our current prototype targets H.264 streams decoded via NVIDIA NVDEC, the dominant codec and hardware path in deployed surveillance and streaming infrastructure. The compressed-domain signals CoStream relies on, motion vectors and frame-type metadata for GOP-aligned processing, are standard primitives shared by all major inter-frame codecs, including H.265/HEVC [54], VP9 [40], and AV1 [11]. Porting CoStream to a new codec requires only extending the front-end Codec Processor to extract these signals, leaving the downstream optimization pipeline intact.

7 Related Work

Compressed-Domain Processing for Video Analytics. Prior work on video analytics has explored query optimization, approximation, cascades, and indexing to reduce the cost of video processing [21, 25, 55, 89]. More closely related to our setting, recent systems have begun to exploit compressed-domain or motion-based signals before full pixel-domain processing. CoVA [23] uses compressed-domain analysis to reduce decoding and inference costs, while Boggart [1] builds motion- and tracking-based indices to accelerate retrospective video analytics. SAND [81] improves the efficiency of GPU-accelerated video preprocessing through better pipeline abstraction and resource reuse. Our work is complementary to these efforts. Rather than optimizing conventional video analytics pipelines alone, CoStream targets online streaming VLM serving, where efficiency depends on jointly coordinating codec-guided visual token pruning with LLM prefilling.

Token Reduction in ViT Encoders. Recent efforts reduce the cost of visual encoding by pruning, merging, or compressing redundant tokens [4, 9, 60, 73, 76, 84, 88, 90, 91]. However, these methods are designed for offline or general multimodal inference and often rely on global video visibility or post-encoding token importance estimation, making them less suitable for causal streaming settings. More recent streaming-oriented approaches adapt token reduction to sequential inputs under causality constraints [10, 24, 79], but still face a nontrivial trade-off among efficiency, temporal awareness, and runtime overhead. In particular, hierarchical token merging and stateful compression often require cross-frame interaction, feature buffering, or extra compression logic, limiting real-time feasibility. Recent work such as CMC [59], Déjà Vu [22], and COPE [56] further shows that codec metadata can expose spatio-temporal sparsity early. However, these systems either reconfigure parameters after feature extraction or encode codec primitives as auxiliary tokens. In contrast, CoStream prunes tokens by mapping codec-derived motion masks directly onto the ViT patch grid before encoding, yielding substantially higher throughput and an order of magnitude lower memory usage.

KVC Management for LLM Decoders. KVC management is critical to efficient LLM serving because it directly affects both memory footprint and decoding throughput [41, 75, 77, 87]. Prior work mainly focuses on system-level memory management [28, 49, 82, 93], cache offloading and reuse [13, 36, 78], and fixed-budget eviction, compression, or quantization [5, 17, 32, 34, 37, 38, 71, 72, 92]. While effective for long-context text and general multimodal workloads, these methods treat KV caches as generic memory objects and thus do not exploit the strong temporal overlap across adjacent windows in streaming video analytics. More recent video-oriented approaches extend these ideas through retrieval, compression, or sparsification of video KVs [16, 41, 50, 57, 75, 77, 87], but still rely on online mechanisms such as cache selection, compression/decompression, retrieval, and external-memory access to identify and manage reusable KV states. In contrast, CoStream directly reuses overlapping-window states already resident in memory and selectively recomputes only drift-sensitive KV states using codec-guided frame-type cues, reducing redundant prefill computation while accelerating processing and lowering memory usage multi-fold.

8 Conclusion

This paper presents CoStream, a codec-guided system for efficient streaming VLM serving. Leveraging codec metadata as a low-cost runtime signal, CoStream jointly optimizes video-side transmission and decoding, codec-guided visual token reduction, and KVC refresh within LLM to reduce redundant computation across the whole serving pipeline. Experimental results show that CoStream significantly reduces end-to-end latency and the required GPU computation while preserving accuracy with negligible runtime overhead.

Acknowledgments

This work was supported in part by Nanyang Technological University, Singapore [grant details here].

References

[1] Neil Agarwal and Ravi Netravali. 2023. Boggart: Towards General-Purpose Acceleration of Retrospective Video Analytics.. In *Proceedings of the 20th Symposium on Networked Systems Design and Implementation (NSDI)*. 933–951.

[2] Asma Baobaid and Mahmoud Méribout. 2025. Edge-GPU Based Face Tracking for Face Detection and Recognition Acceleration. *CoRR* abs/2505.04524 (2025).

[3] Gedas Bertasius, Heng Wang, and Lorenzo Torresani. 2021. Is Space-Time Attention All You Need for Video Understanding?. In *The 38th Annual Conference on Machine Learning (ICML)*. 813–824.

[4] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. 2023. Token Merging: Your ViT But Faster.. In *The International Conference on Learning Representations 2023 (ICLR)*.

[5] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. 2024. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. *CoRR* abs/2406.02069 (2024).

[6] João Carreira and Andrew Zisserman. 2017. Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset.. In *The IEEE/CVF Conference on Computer Vision and Pattern Recognition 2017 (CVPR)*. 4724–4733.

[7] Amine Chaabouni, Yann Gaudeau, Julien Lambert, J-M Moureaux, and Patrice Gallet. 2016. H. 264 medical video compression for telemedicine: A performance analysis. *IRBM* 37, 1 (2016), 40–48.

[8] Joya Chen, Ziyun Zeng, Yiqi Lin, Wei Li, Zejun Ma, and Mike Zheng Shou. 2025. LiveCC: Learning Video LLM with Streaming Speech Transcription at Scale.. In *The IEEE/CVF Conference on Computer Vision and Pattern Recognition 2025 (CVPR)*. 29083–29095.

[9] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. 2024. An Image is Worth 1/2 Tokens After Layer 2: Plug-and-Play Inference Acceleration for Large Vision-Language Models.. In *ECCV (81)*. 19–35.

[10] Xueyi Chen, Keda Tao, Kele Shao, and Huan Wang. 2025. StreamingTOM: Streaming Token Compression for Efficient Video Understanding. *CoRR* abs/2510.18269 (2025).

[11] Yue Chen, Debargha Mukherjee, Jingning Han, Adrian Grange, Yaowu Xu, Zoe Liu, Sarah Parker, Cheng Chen, Hui Su, Urvang Joshi, Ching-Han Chiang, Yunqing Wang, Paul Wilkins, Jim Bankoski, Luc N. Trudeau, Nathan E. Egge, Jean-Marc Valin, Thomas Davies, Steinar Midtskogen, Andrey Norkin, and Peter De Rivaz. 2018. An Overview of Core Coding Tools in the AV1 Video Codec. 41–45.

[12] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, Bin Li, Ping Luo, Tong Lu, Yu Qiao, and Jifeng Dai. 2023. InternVL: Scaling up Vision Foundation Models and Aligning for Generic Visual-Linguistic Tasks. *CoRR* abs/2312.14238 (2023).

[13] Yihua Cheng, Yuhan Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. 2025. LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. *CoRR* abs/2510.09665 (2025).

[14] Comparitech. 2025. Surveillance Camera Statistics: Which City has the Most CCTV? https://www.comparitech.com/vpn-privacy/the-worlds-most-surveilled-cities/. Accessed 2026-03-26.

[15] Khaled Waleed Dawoud, Zaigham Zaheer, Mustaqeem Khan, Karthik Nandakumar, Abdulmotaleb Elsaddik, and Muhammad Haris Khan. 2025. FusedVision: A Knowledge-Infusing Approach for Practical Anomaly Detection in Real-world Surveillance Videos.. In *CVPR Workshops*. 4036–4046.

[16] Shangzhe Di, Zhelun Yu, Guanghao Zhang, Haoyuan Li, Tao Zhong, Hao Cheng, Bolin Li, Wanggui He, Fangxun Shu, and Hao Jiang. 2025. Streaming Video Question-Answering with In-context Video

KV-Cache Retrieval.. In *The International Conference on Learning Representations 2025 (ICLR)*.

[17] Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. 2024. Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. *CoRR* abs/2407.11550 (2024).

[18] Edward Fish and Andrew Gilbert. 2025. PLOT-TAL: Prompt-Learning with Optimal Transport for Few-Shot Temporal Action Localization.. In *The International Conference on Computer Vision Workshops 2025 (ICCVW)*. 5912–5921.

[19] Ibrahim Ethem Hamamci, Sezgin Er, Suprosanna Shit, Hadrien Reynaud, Dong Yang, Pengfei Guo, Marc Edgar, Daguang Xu, Bernhard Kainz, and Bjoern H. Menze. 2025. Better Tokens for Better 3D: Advancing Vision-Language Modeling in 3D Medical Imaging. *CoRR* abs/2510.20639 (2025).

[20] Zelin He, Sarah Alnegheimish, and Matthew Reimherr. 2025. Harnessing Vision-Language Models for Time Series Anomaly Detection. *CoRR* abs/2506.06836 (2025).

[21] Kevin Hsieh, Ganesh Ananthanarayanan, Peter Bodík, Shivaram Venkataraman, Paramvir Bahl, Matthai Philipose, Phillip B. Gibbons, and Onur Mutlu. 2018. Focus: Querying Large Video Datasets with Low Latency and Low Cost.. In *Proceedings of the 13th Symposium on Operating System Design and Implementation (OSDI)*. 269–286.

[22] Jinwoo Hwang, Daeun Kim, Sangyeop Lee, Yoonsung Kim, Guseul Heo, Hojoon Kim, Yunseok Jeong, Tadiwos Meaza, Eunhyeok Park, Jeongseob Ahn, and Jongse Park. 2025. Déjà Vu: Efficient Video-Language Query Engine with Learning-based Inter-Frame Computation Reuse. *Proc. VLDB Endow.* 18, 10 (2025), 3284–3298.

[23] Jinwoo Hwang, Minsu Kim, Daeun Kim, Seungho Nam, Yoonsung Kim, Dohee Kim, Hardik Sharma, and Jongse Park. 2022. CoVA: Exploiting Compressed-Domain Analysis to Accelerate Video Analytics.. In *Proceedings of the 2022 USENIX Annual Technical Conference (ATC)*. 707–722.

[24] Jindong Jiang, Amala Sanjay Deshmukh, Kateryna Chumachenko, Karan Sapra, Zhiding Yu, Guilin Liu, Andrew Tao, Pavlo Molchanov, Jan Kautz, and Wonmin Byeon. 2026. Stateful Token Reduction for Long-Video Hybrid VLMs. *arXiv preprint arXiv:2603.00198* (2026).

[25] Daniel Kang, John Emmons, Firas Abuzaid, Peter Bailis, and Matei Zaharia. 2017. Noscope: optimizing neural network queries over video at scale. *arXiv preprint arXiv:1703.02529* (2017).

[26] Brendan Klare and Mark Burge. 2010. Assessment of H. 264 video compression on automated face recognition performance in surveillance and mobile video scenarios. In *Biometric Technology for Human Identification VII*, Vol. 7667. SPIE, 325–332.

[27] Vaibhav Kurrey, Sivakalyan Pujari, and Gagan Raj Gupta. 2025. Process Integrated Computer Vision for Real-Time Failure Prediction in Steel Rolling Mill. *CoRR* abs/2510.26684 (2025).

[28] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention.. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*. 611–626.

[29] Seon-Ho Lee, Jue Wang, Zhikang Zhang, David Fan, and Xinyu Li. 2024. Video Token Merging for Long Video Understanding.. In *The 38th Annual Conference on Neural Information Processing Systems (NeurIPS)*.

[30] Feng Li, Renrui Zhang, Hao Zhang, Yuanhan Zhang, Bo Li, Wei Li 0190, Zejun Ma, and Chunyuan Li. 2024. LLaVA-NeXT-Interleave: Tackling Multi-image, Video, and 3D in Large Multimodal Models. *CoRR* abs/2407.07895 (2024).

[31] Yifan Li, Wentao Bao, Botao Ye, Zhen Tan, Tianlong Chen, Huan Liu, and Yu Kong. 2025. Window Token Concatenation for Efficient Visual Large Language Models.. In *CVPR Workshops*. 3187–3197.

[32] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You are Looking for Before Generation.. In *The 38th Annual Conference on Neural Information Processing Systems (NeurIPS)*.

[33] Yuanqi Li, Arthi Padmanabhan, Pengzhan Zhao, Yufei Wang, Guoqing Harry Xu, and Ravi Netravali. 2020. Reducto: On-Camera Filtering for Resource-Efficient Real-Time Video Analytics.. In *Proceedings of the ACM SIGCOMM 2020 Conference*. 359–376.

[34] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Reza Haffari, and Bohan Zhuang. 2024. MiniCache: KV Cache Compression in Depth Dimension for Large Language Models.. In *The 38th Annual Conference on Neural Information Processing Systems (NeurIPS)*.

[35] Jihao Liu, Zhiding Yu, Shiyi Lan, Shihao Wang, Rongyao Fang, Jan Kautz, Hongsheng Li, and José M. Álvarez. 2024. StreamChat: Chatting with Streaming Video. *CoRR* abs/2412.08646 (2024).

[36] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving.. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.

[37] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. 2023. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time.. In *The 37th Annual Conference on Neural Information Processing Systems (NeurIPS)*.

[38] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen (Henry) Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache.. In *The 41st Annual Conference on Machine Learning (ICML)*. 32332–32344.

[39] Market Reports World. 2026. CCTV Cameras Market Size, Share, Growth, and Industry Analysis, Forecast to 2034. https://www.marketreportsworld.com/market-reports/cctv-cameras-market-14721988

[40] Debargha Mukherjee, Jim Bankoski, Adrian Grange, Jingning Han, John Koleszar, Paul Wilkins, Yaowu Xu, and Ronald Bultje. 2013. The latest open-source video codec VP9 - An overview and preliminary results. In *2013 Picture Coding Symposium (PCS)*. https://doi.org/10.1109/PCS.2013.6737765

[41] Zhenyu Ning, Guangda Liu, Qihao Jin, Wenchao Ding, Minyi Guo, and Jieru Zhao. 2025. LiveVLM: Efficient Online Video Understanding via Streaming-Oriented KV Cache and Retrieval. *CoRR* abs/2505.15269 (2025).

[42] Junbo Niu, Yifei Li, Ziyang Miao, Chunjiang Ge, Yuanhang Zhou, Qihao He, Xiaoyi Dong, Haodong Duan, Shuangrui Ding, Rui Qian, Pan Zhang, Yuhang Zang, Yuhang Cao, Conghui He, and Jiaqi Wang. 2025. OVO-Bench: How Far is Your Video-LLMs from Real-World Online Video Understanding?. In *The IEEE/CVF Conference on Computer Vision and Pattern Recognition 2025 (CVPR)*. 18902–18913.

[43] NVIDIA. 2025. Europe Builds AI Infrastructure With NVIDIA to Fuel Region's Next Industrial Transformation. https://nvidianews.nvidia.com/news/europe-ai-infrastructure. Accessed 2026-03-26.

[44] NVIDIA. 2025. NVIDIA and Partners Build America's AI Infrastructure and Create Blueprint to Power the Next Industrial Revolution. https://nvidianews.nvidia.com/news/nvidia-partners-ai-infrastructure-america. Accessed 2026-03-26.

[45] NVIDIA. 2026. VDEC Application Note. https://docs.nvidia.com/video-technologies/video-codec-sdk/13.0/nvdec-application-note/index.html

[46] Tsung-Yin Ou, Andrés Ponce, Cody Lee, and Areoll Wu. 2025. Real-time retail planogram compliance application using computer vision and virtual shelves. *Scientific Reports* 15, 1 (2025), 43898.

[47] Junting Pan, Ziyi Lin, Xiatian Zhu, Jing Shao, and Hongsheng Li. 2022. ST-Adapter: Parameter-Efficient Image-to-Video Transfer Learning.. In *The 36th Annual Conference on Neural Information Processing Systems (NeurIPS)*.

[48] Persistence Market Research. 2024. CCTV Cameras Market Size, Share & Forecast to 2032. https://www.persistencemarketresearch.com/market-research/cctv-cameras-market.asp

[49] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2024. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. *CoRR* abs/2405.04437 (2024).

[50] Minghao Qin, Xiangrui Liu, Zhengyang Liang, Yan Shu, Huaying Yuan, Junjie Zhou, Shitao Xiao, Bo Zhao, and Zheng Liu. 2025. Video-XL-2: Towards Very Long-Video Understanding Through Task-Aware KV Sparsification. *CoRR* abs/2506.19225 (2025).

[51] Shengling Qin, Hao Yu, Chenxin Wu, Zheng Li, Yizhong Cao, Zhengyang Zhuge, Yuxin Zhou, Wentao Yao, Yi Zhang, Zhengheng Wang, Shuai Bai, Jianwei Zhang, and Junyang Lin. 2025. VLCache: Computing 2% Vision Tokens and Reusing 98% for Vision-Language Inference. *CoRR* abs/2512.12977 (2025).

[52] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. 2021. DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification.. In *The 35th Annual Conference on Neural Information Processing Systems (NeurIPS)*. 13937–13949.

[53] António Gouveia Ribeiro, Luís Vilaça, Carlos Costa, Tiago Soares da Costa, and Pedro Miguel Carvalho. 2025. Automatic Visual Inspection for Industrial Application. *J. Imaging* 11, 10 (2025), 350.

[54] Iain E Richardson. 2024. *Coding video: A practical guide to HEVC and beyond*. John Wiley & Sons.

[55] Francisco Romero, Johann Hauswald, Aditi Partap, Daniel Kang, Matei Zaharia, and Christos Kozyrakis. 2022. Optimizing Video Analytics with Declarative Model Relationships. *Proc. VLDB Endow.* 16, 3 (2022), 447–460.

[56] Sayan Deb Sarkar, Rémi Pautrat, Ondrej Miksik, Marc Pollefeys, Iro Armeni, Mahdi Rad, and Mihai Dusmanu. 2026. CoPE-VideoLM: Codec Primitives For Efficient Video Language Models. *CoRR* abs/2602.13191 (2026).

[57] Benjamin Schneider, Dongfu Jiang, Chao Du, Tianyu Pang, and Wenhu Chen. 2025. QuickVideo: Real-Time Long Video Understanding with System Algorithm Co-Design. *CoRR* abs/2505.16175 (2025).

[58] Akash Sharma, Pranjal Naman, Roopkatha Banerjee, Priyanshu Pansari, Sankalp Gawali, Mayank Arya, Sharath Chandra, Arun Josephraj, Rakshit Ramesh, Punit Rathore, et al. 2026. Scaling Real-Time Traffic Analytics on Edge-Cloud Fabrics for City-Scale Camera Networks. *arXiv preprint arXiv:2603.05217* (2026).

[59] Zhuoran Song, Chunyu Qi, Fangxin Liu, Naifeng Jing, and Xiaoyao Liang. 2024. CMC: Video Transformer Acceleration via CODEC Assisted Matrix Condensing.. In *ASPLOS (2)*. 201–215.

[60] Keda Tao, Can Qin, Haoxuan You, Yang Sui, and Huan Wang. 2025. DyCoke: Dynamic Compression of Tokens for Fast Video Large Language Models.. In *The IEEE/CVF Conference on Computer Vision and Pattern Recognition 2025 (CVPR)*. 18992–19001.

[61] LMCache Team. 2026. LMCache. https://github.com/lmcache/lmcache

[62] Qwen Team. 2025. Qwen3 Technical Report. *CoRR* abs/2505.09388 (2025).

[63] Qwen Team. 2025. Qwen3-VL Technical Report. *CoRR* abs/2511.21631 (2025).

[64] vLLM Team. 2026. Easy, fast, and cheap LLM serving for everyone. https://github.com/vllm-project/vllm

[65] Limin Wang, Yuanjun Xiong, Zhe Wang, Yu Qiao, Dahua Lin, Xiaoou Tang, and Luc Van Gool. 2016. Temporal Segment Networks: Towards Good Practices for Deep Action Recognition.. In *ECCV (8)*. 20–36.

[66] Qinsi Wang, Hancheng Ye, Ming-Yu Chung, Yudong Liu, Yueqian Lin, Martin Kuo, Mingyuan Ma, Jianyi Zhang, and Yiran Chen. 2025. CoreMatching: A Co-adaptive Sparse Inference Framework with Token and Neuron Pruning for Comprehensive Acceleration of Vision-Language Models.. In *The 42nd Annual Conference on Machine Learning (ICML)*.

[67] Thomas Wiegand, Gary J. Sullivan, Gisle Bjøntegaard, and Ajay Luthra. 2003. Overview of the H.264/AVC video coding standard. *IEEE Trans. Circuits Syst. Video Technol.* 13, 7 (2003), 560–576.

[68] Jiangkai Wu, Liming Liu, Yunpeng Tan, Junlin Hao, and Xinggong Zhang. 2024. Promptus: Can Prompts Streaming Replace Video Streaming with Stable Diffusion. *CoRR* abs/2405.20032 (2024).

[69] Jiang Wu, Sichao Wu, Yinsong Ma, Guangyuan Yu, Haoyuan Xu, Lifang Zheng, and Jingliang Duan. 2025. MonitorVLM:A Vision Language Framework for Safety Violation Detection in Mining Operations. *CoRR* abs/2510.03666 (2025).

[70] Zhiyu Wu, Xiaokang Chen, Zizheng Pan, Xingchao Liu, Wen Liu, Damai Dai, Huazuo Gao, Yiyang Ma, Chengyue Wu, Bingxuan Wang, Zhenda Xie, Yu Wu, Kai Hu, Jiawei Wang, Yaofeng Sun, Yukun Li, Yishi Piao, Kang Guan, Aixin Liu, Xin Xie, Yuxiang You, Kai Dong, Xingkai Yu, Haowei Zhang, Liang Zhao, Yisong Wang, and Chong Ruan. 2024. DeepSeek-VL2: Mixture-of-Experts Vision-Language Models for Advanced Multimodal Understanding. *CoRR* abs/2412.10302 (2024).

[71] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. 2024. DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. *CoRR* abs/2410.10819 (2024).

[72] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient Streaming Language Models with Attention Sinks. *CoRR* abs/2309.17453 (2023).

[73] Long Xing, Qidong Huang, Xiaoyi Dong, Jiajie Lu, Pan Zhang, Yuhang Zang, Yuhang Cao, Conghui He, Jiaqi Wang, Feng Wu, and Dahua Lin. 2024. PyramidDrop: Accelerating Your Large Vision-Language Models via Pyramid Visual Redundancy Reduction. *CoRR* abs/2410.17247 (2024).

[74] Chenting Xu, Ke Xu, Xinghao Jiang, and Tanfeng Sun. 2025. PLOVAD: Prompting Vision-Language Models for Open Vocabulary Video Anomaly Detection. *IEEE Trans. Circuits Syst. Video Technol.* 35, 6 (2025), 5925–5938.

[75] Ruyi Xu, Guangxuan Xiao, Yukang Chen, Liuning He, Kelly Peng, Yao Lu, and Song Han. 2025. StreamingVLM: Real-Time Understanding for Infinite Video Streams. *CoRR* abs/2510.09608 (2025).

[76] Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. 2025. VisionZip: Longer is Better but Not Necessary in Vision Language Models.. In *The IEEE/CVF Conference on Computer Vision and Pattern Recognition 2025 (CVPR)*. 19792–19802.

[77] Yanlai Yang, Zhuokai Zhao, Satya Narayan Shukla, Aashu Singh, Shlok Kumar Mishra, Lizhu Zhang, and Mengye Ren. 2025. StreamMem: Query-Agnostic KV Cache Memory for Streaming Video Understanding. *CoRR* abs/2508.15717 (2025).

[78] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion.. In *Proceedings of the 2025 EuroSys Conference*. 94–109.

[79] Linli Yao, Yicheng Li, Yuancheng Wei, Lei Li, Shuhuai Ren, Yuanxin Liu, Kun Ouyang, Lean Wang, Shicheng Li, Sida Li, Lingpeng Kong, Qi Liu, Yuanxing Zhang, and Xu Sun. 2025. TimeChat-Online: 80% Visual Tokens are Naturally Redundant in Streaming Videos.. In *ACM Multimedia*. 10807–10816.

[80] Shanle Yao, Narges Rashvand, Armin Danesh Pazho, and Hamed Tabkhi. 2026. From Offline to Periodic Adaptation for Pose-Based Shoplifting Detection in Real-world Retail Security. *IEEE Internet of Things Journal* (2026).

[81] Juncheol Ye, Seungkook Lee, Hwijoon Lim, Jihyuk Lee, Uitaek Hong, Youngjin Kwon, and Dongsu Han. 2025. SAND: A New Programming Abstraction for Video-based Deep Learning.. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP)*. 589–605.

[82] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. ChunkAttention: Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition.. In *ACL (1)*. 11608–11620.

[83] Muchao Ye, Weiyang Liu, and Pan He. 2025. VERA: Explainable Video Anomaly Detection via Verbalized Learning of Vision-Language Models.. In *The IEEE/CVF Conference on Computer Vision and Pattern Recognition 2025 (CVPR)*. 8679–8688.

[84] Weihao Ye, Qiong Wu, Wenhao Lin, and Yiyi Zhou. 2025. Fit and Prune: Fast and Training-free Visual Token Pruning for Multi-modal Large Language Models.. In *The 39th Annual AAAI Conference on Artificial Intelligence (AAAI)*. 22128–22136.

[85] Tongtong Yuan, Xuange Zhang, Kun Liu, Bo Liu, Chen Chen, Jian Jin, and Zhenzhen Jiao. 2024. Towards Surveillance Video-and-Language Understanding: New Dataset, Baselines, and Challenges.. In *The IEEE/CVF Conference on Computer Vision and Pattern Recognition 2024 (CVPR)*. 22052–22061.

[86] Luca Zanella, Willi Menapace, Massimiliano Mancini, Yiming Wang, and Elisa Ricci. 2024. Harnessing Large Language Models for Training-Free Video Anomaly Detection.. In *The IEEE/CVF Conference on Computer Vision and Pattern Recognition 2024 (CVPR)*. 18527–18536.

[87] Xiangyu Zeng, Kefan Qiu, Qingyu Zhang, Xinhao Li, Jing Wang, Jiaxin Li, Ziang Yan, Kun Tian, Meng Tian, Xinhai Zhao, Yi Wang, and Limin Wang. 2025. StreamForest: Efficient Online Video Understanding with Persistent Event Memory. *CoRR* abs/2509.24871 (2025).

[88] Ce Zhang, Kaixin Ma, Tianqing Fang, Wenhao Yu, Hongming Zhang, Zhisong Zhang, Yaqi Xie, Katia P. Sycara, Haitao Mi, and Dong Yu. 2025. VScan: Rethinking Visual Token Reduction for Efficient Large Vision-Language Models. *CoRR* abs/2505.22654 (2025).

[89] Haoyu Zhang, Ganesh Ananthanarayanan, Peter Bodík, Matthai Philipose, Paramvir Bahl, and Michael J. Freedman. 2017. Live Video Analytics at Scale with Approximation and Delay-Tolerance.. In *Proceedings of the 14th Symposium on Networked Systems Design and Implementation (NSDI)*. 377–392.

[90] Qizhe Zhang, Aosong Cheng, Ming Lu, Renrui Zhang, Zhiyong Zhuo, Jiajun Cao, Shaobo Guo, Qi She, and Shanghang Zhang. 2025. Beyond text-visual attention: Exploiting visual cues for effective token pruning in vlms. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 20857–20867.

[91] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis A. Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, and Shanghang Zhang. 2025. SparseVLM: Visual Token Sparsification for Efficient Vision-Language Model Inference.. In *The 42nd Annual Conference on Machine Learning (ICML)*.

[92] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark W. Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models.. In *The 37th Annual Conference on Neural Information Processing Systems (NeurIPS)*.

[93] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs.. In *The 38th Annual Conference on Neural Information Processing Systems (NeurIPS)*.

[94] Wei Zhou, Li Yang, Lei Zhao, Runyu Zhang, Yifan Cui, Hongpu Huang, Kun Qie, and Chen Wang. 2026. Vision Technologies with Applications in Traffic Surveillance Systems: A Holistic Survey. *ACM Comput. Surv.* 58, 3 (2026), 58:1–58:47.

[95] Jinguo Zhu, Weiyun Wang, Zhe Chen, Zhaoyang Liu, Shenglong Ye, Lixin Gu, Hao Tian, Yuchen Duan, Weijie Su, Jie Shao, Zhangwei Gao, Erfei Cui, Xuehui Wang, Yue Cao, Yangzhou Liu, Xingguang Wei, Hongjie Zhang, Haomin Wang, Weiye Xu, Hao Li, Jiahao Wang, Nianchen Deng, Songze Li, Yinan He, Tan Jiang, Jiapeng Luo, Yi Wang, Conghui He, Botian Shi, Xingcheng Zhang, Wenqi Shao, Junjun He, Yingtong Xiong, Wenwen Qu, Peng Sun, Penglong Jiao, Han Lv, Lijun Wu, Kaipeng Zhang, Huipeng Deng, Jiaye Ge, Kai Chen, Limin Wang, Min Dou, Lewei Lu, Xizhou Zhu, Tong Lu, Dahua Lin, Yu Qiao, Jifeng Dai, and Wenhai Wang. 2025. InternVL3: Exploring Advanced Training and Test-Time Recipes for Open-Source Multimodal Models. *CoRR* abs/2504.10479 (2025).